ClearLedger: Архитектура хранилища и обработки данных

> **Дата:** 2026-02-27 (обновлено)
> **Статус:** Зафиксированное архитектурное решение (ADR)
> **Контекст:** Трёхуровневая архитектура данных с разделением обработки между клиентом и облаком

---

0. Маппинг бизнес-кейсов на архитектурные слои

```
Кейс 1 (Приём + верификация)     → Layer 1 (RAW) + Layer 1a (staging) + Layer
2 (public)
                                      Pipeline: detect → extract → classify →
dedup → verify
                                      Эталон: справочники НСИ из 1С
(контрагенты, договоры, номенклатура)
                                      Внешние сервисы: ФНС, Rusprofile,
судебные базы

Кейс 2 (AI-анализ периода)       → Layer 2 (сверки) + Cloud AI (Skills)
                                      Skills: reconciliation/*.md — правила
сверки
                                      Непрерывный анализ: event-driven, по мере
поступления документов
                                      Автосверки с контрагентами: акты сверки,
расхождения

Кейс 3 (Моделирование закрытия)  → Layer 2 → Layer 3 (accounting_documents) +
Accounting Kernel
                                      Песочница: виртуальное закрытие периода
                                      3 контура: управленческий, бухгалтерский,
налоговый
                                      Skills: accounting/*.md + contracts/*.md
```

---

1. Общая схема

```
Контейнер клиента (on-premise или наше облако)
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  UI (React SPA)  ↔  API (FastAPI)  ↔  PostgreSQL         │
│                           ↕                              │
│  Layer 1: /data/storage/  (оригиналы файлов, immutable)  │
│  Layer 1a: schema staging (черновики, скрыт от клиента)  │
│  Layer 2: schema public   (чистые данные, клиент видит)  │
│                                                          │
└──────────────────────────────────────────────────────────┘
                             │
                             │ sync (когда есть интернет)
                             ▼
                   Наш облачный сервер
                 ┌──────────────────────┐
                 │  AI-обработка:       │
                 │  классификация       │
                 │  нормализация        │
                 │  валидация           │
                 │  отбраковка          │
                 └──────────────────────┘
```

---

2. Модель поставки

Один Docker-образ `clearledger:latest`. Клиент выбирает, где запускать:

```
clearledger:latest
├── docker-compose up    → на машине клиента (on-premise)
└── docker-compose up    → на нашем сервере (облако, отдельный instance)
```

Нет мультитенантности в коде. Один instance = один клиент. Изоляция — на уровне контейнера.

---

3. Три логических слоя данных

Layer 1 — RAW (оригиналы файлов)

Что: PDF, сканы, Excel, XML, Word, email — как загрузил клиент.
Где: Файловая система `/data/storage/` в папочной структуре.
Природа: Immutable. Записывается один раз, не изменяется.

Зачем: Источник правды. Доказательство для аудита. Можно перепарсить при смене алгоритмов.

```
/data/storage/
  {company_id}/
    {year}/
      {month}/
        {category_id}/
          {subcategory_id}/
            {source_uuid}_{original_filename}
```

Пример:

```
/data/storage/
  npk/
    2026/
      02/
        primary/
          ttn/
            a1b2c3d4_TTH_245.pdf
            e5f6g7h8_TTH_246.pdf
          acts/
            i9j0k1l2_Акт_сверки_январь.pdf
        financial/
          payments/
            m3n4o5p6_Платежка_1234.pdf
```

Клиент может подмонтировать `/data/storage` как сетевой диск и видеть файлы в проводнике.

Layer 1a — Staging (рабочая зона, скрыта от клиента)

Что: Результаты первичного парсинга, черновики записей, очередь на AI-обработку, результаты обработки, отбраковка.
Где: PostgreSQL, schema `staging`.
Природа: Мутабельная. Данные обновляются по мере прохождения обработки.
Доступ: Только API внутренними эндпоинтами. Клиентский UI не видит.

```
-- ============================================================
-- SCHEMA: staging (Layer 1a — внутренняя кухня)
-- ============================================================
```

```sql
CREATE SCHEMA staging;

-- Черновики записей (результат локального парсинга)
CREATE TABLE staging.raw_entries (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id      TEXT NOT NULL,
    source_id       UUID NOT NULL,               -- ссылка на файл в Layer 1
    file_name       TEXT NOT NULL,
    mime_type       TEXT NOT NULL,
    -- Результат локального парсинга
    extracted_text  TEXT NOT NULL DEFAULT '',
    extracted_fields JSONB NOT NULL DEFAULT '{}',
        -- { "docNumber": "245", "docDate": "2026-02-25", ... }
    page_count      INT,
    -- Статус обработки
    processing_status TEXT NOT NULL DEFAULT 'parsed',
        -- parsed      = локальный парсинг выполнен, ждёт отправки в облако
        -- sent        = отправлено в облако
        -- processed   = облако вернуло результат
        -- promoted    = перемещено в Layer 2 (public.entries)
        -- rejected    = отвергнуто AI
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_raw_entries_status ON staging.raw_entries(processing_status);

-- Результаты AI-обработки (приходят из облака)
CREATE TABLE staging.ai_results (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    raw_entry_id    UUID NOT NULL REFERENCES staging.raw_entries(id) ON DELETE
CASCADE,
    -- Классификация
    category_id     TEXT,
    subcategory_id  TEXT,
    doc_type_id     TEXT,
    confidence      REAL NOT NULL DEFAULT 0,
    -- Нормализованные метаданные
    normalized_metadata JSONB NOT NULL DEFAULT '{}',
        -- { "docNumber": "245", "docDate": "2026-02-25",
        --   "counterparty": "ООО Лукойл", "counterparty_inn": "7707083893",
        --   "amount": "65100.00", "currency": "RUB" }
    metadata_version TEXT NOT NULL DEFAULT 'v1',
        -- Версия schema нормализованных метаданных (согласована с
public.entries.metadata_version)
```

```
    -- Решение
    decision        TEXT NOT NULL DEFAULT 'pending',
        -- accepted          = принято, готово к промоушену в Layer 2
        -- needs_review      = требует ручной проверки
        -- reclassified      = переклассифицировано (другая категория)
        -- rejected          = отвергнуто (не валидный документ)
        -- duplicate         = дубликат существующего
    rejection_reason TEXT,
    duplicate_of    UUID,                       -- если дубликат — ссылка на
оригинал
    -- Лог
    processed_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    model_version   TEXT                        -- версия модели, которая
обработала
);

CREATE INDEX idx_ai_results_entry ON staging.ai_results(raw_entry_id);
CREATE INDEX idx_ai_results_decision ON staging.ai_results(decision);

-- Очередь синхронизации с облаком
CREATE TABLE staging.sync_queue (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    raw_entry_id    UUID NOT NULL REFERENCES staging.raw_entries(id) ON DELETE
CASCADE,
    direction       TEXT NOT NULL,              -- to_cloud, from_cloud
    payload         JSONB NOT NULL DEFAULT '{}',
    status          TEXT NOT NULL DEFAULT 'pending',
        -- pending, sent, completed, failed
    attempts        INT NOT NULL DEFAULT 0,
    last_attempt    TIMESTAMPTZ,
    error           TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_sync_queue_status ON staging.sync_queue(status);
```

Layer 2 — Public (чистые данные, клиент видит)

Что: Нормализованные записи, прошедшие AI-обработку и валидацию. Готовы для дашбордов, Excel, сводных таблиц, 1С.
Где: PostgreSQL, schema `public`.
Природа: Мутабельная (статусы меняются), но данные уже качественные.
Доступ: Клиентский UI работает только с этой схемой.

```sql
-- ================================================================
-- SCHEMA: public (Layer 2 — витрина для клиента)
-- ================================================================

-- Пользователи
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email           TEXT NOT NULL UNIQUE,
    name            TEXT NOT NULL,
    password_hash   TEXT NOT NULL,
    role            TEXT NOT NULL DEFAULT 'operator',
        -- admin    = мы, удалённо
        -- owner    = владелец instance
        -- operator = загрузка, просмотр, верификация
        -- viewer   = только чтение
    is_active       BOOLEAN NOT NULL DEFAULT true,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_login      TIMESTAMPTZ
);

-- Компании (в рамках instance может быть группа)
CREATE TABLE companies (
    id              TEXT PRIMARY KEY,
    name            TEXT NOT NULL,
    short_name      TEXT NOT NULL,
    inn             TEXT,
    profile_id      TEXT NOT NULL,               -- fuel, trade, retail,
energy, general
    color           TEXT NOT NULL DEFAULT '#3b82f6',
    settings        JSONB NOT NULL DEFAULT '{}',
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE user_companies (
    user_id         UUID REFERENCES users(id) ON DELETE CASCADE,
    company_id      TEXT REFERENCES companies(id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, company_id)
);

-- Источники (метаданные файлов, сами файлы — на диске Layer 1)
CREATE TABLE sources (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id      TEXT NOT NULL REFERENCES companies(id),
    file_name       TEXT NOT NULL,
    mime_type       TEXT NOT NULL,
    file_size       BIGINT NOT NULL,
```

```sql
    file_path       TEXT NOT NULL,              -- путь в /data/storage/
    fingerprint     TEXT NOT NULL,              -- SHA-256
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_sources_company ON sources(company_id);
CREATE INDEX idx_sources_fingerprint ON sources(fingerprint);

-- Записи (основная бизнес-сущность, ТОЛЬКО чистые данные)
CREATE TABLE entries (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id      TEXT NOT NULL REFERENCES companies(id),
    source_id       UUID REFERENCES sources(id),
    title           TEXT NOT NULL,
    category_id     TEXT NOT NULL,
    subcategory_id  TEXT NOT NULL,
    doc_type_id     TEXT,
    status          TEXT NOT NULL DEFAULT 'new',
        -- new → verified → transferred → error
    source          TEXT NOT NULL,
        -- upload, photo, manual, api, email, oneC, paste
    source_label    TEXT NOT NULL DEFAULT '',
    metadata        JSONB NOT NULL DEFAULT '{}',
        -- Нормализованные метаданные (прошли AI-обработку)
    metadata_version TEXT NOT NULL DEFAULT 'v1',
        -- Версия schema метаданных. При изменении структуры metadata
        -- нормализатор проверяет версию и мигрирует при необходимости.
        -- Позволяет читать старые записи без потери совместимости.

    -- === Accounting-critical generated columns ===
    -- Ключевые учётные поля вынесены из JSONB в типизированные колонки.
    -- Автоматически вычисляются из metadata, индексируются, имеют
constraints.
    -- Это гарантирует целостность на уровне БД, а не только Pydantic.
    doc_number      TEXT GENERATED ALWAYS AS (metadata->>'docNumber') STORED,
    doc_date        DATE GENERATED ALWAYS AS ((metadata->>'docDate')::date)
STORED,
    counterparty_inn TEXT GENERATED ALWAYS AS (metadata->>'counterparty_inn')
STORED,
    total_amount    NUMERIC GENERATED ALWAYS AS (
        CASE WHEN metadata->>'totalAmount' IS NOT NULL
             THEN (metadata->>'totalAmount')::numeric
             ELSE NULL END
    ) STORED,
    vat_amount      NUMERIC GENERATED ALWAYS AS (
        CASE WHEN metadata->>'vatAmount' IS NOT NULL
```

```sql
            THEN (metadata->>'vatAmount')::numeric
            ELSE NULL END
    ) STORED,

    -- Constraints на accounting-critical поля
    CONSTRAINT chk_total_amount CHECK (total_amount IS NULL OR total_amount >=
0),
    CONSTRAINT chk_vat_amount CHECK (vat_amount IS NULL OR vat_amount >= 0),

    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    verified_at     TIMESTAMPTZ,
    verified_by     UUID REFERENCES users(id),
    transferred_at  TIMESTAMPTZ
);

CREATE INDEX idx_entries_company ON entries(company_id);
CREATE INDEX idx_entries_status ON entries(company_id, status);
CREATE INDEX idx_entries_category ON entries(company_id, category_id);
CREATE INDEX idx_entries_created ON entries(company_id, created_at DESC);
CREATE INDEX idx_entries_metadata ON entries USING GIN (metadata
jsonb_path_ops);
CREATE INDEX idx_entries_doc_date ON entries(company_id, doc_date DESC);
CREATE INDEX idx_entries_counterparty ON entries(counterparty_inn) WHERE
counterparty_inn IS NOT NULL;

-- Коннекторы
CREATE TABLE connectors (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id      TEXT NOT NULL REFERENCES companies(id),
    name            TEXT NOT NULL,
    type            TEXT NOT NULL,
    url             TEXT NOT NULL DEFAULT '',
    config          JSONB NOT NULL DEFAULT '{}',
    status          TEXT NOT NULL DEFAULT 'disabled',
    category_id     TEXT NOT NULL,
    interval_sec    INT NOT NULL DEFAULT 3600,
    last_sync       TIMESTAMPTZ,
    records_count   INT NOT NULL DEFAULT 0,
    errors_count    INT NOT NULL DEFAULT 0,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Аудит
CREATE TABLE audit_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    user_id         UUID REFERENCES users(id),
    action          TEXT NOT NULL,
    entity_type     TEXT NOT NULL,
    entity_id       TEXT,
    details         JSONB NOT NULL DEFAULT '{}',
    ip_address      INET,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_entity ON audit_log(entity_type, entity_id);
CREATE INDEX idx_audit_created ON audit_log(created_at DESC);

-- Настройки
CREATE TABLE settings (
    key             TEXT PRIMARY KEY,
    value           JSONB NOT NULL,
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

---

4. Что входит в контейнер

```yaml
# docker-compose.yml
services:
  web:
    image: clearledger
    ports: ["443:443", "80:80"]
    volumes:
      - storage:/data/storage
      - backups:/data/backups
    environment:
      - DATABASE_URL=postgresql://cl:${DB_PASS}@db:5432/clearledger
      - STORAGE_ROOT=/data/storage
      - SECRET_KEY=${SECRET_KEY}
      - CLOUD_API_URL=${CLOUD_API_URL}     # URL нашего облачного сервера
      - CLOUD_API_KEY=${CLOUD_API_KEY}     # ключ авторизации к облаку
      - INSTANCE_ID=${INSTANCE_ID}         # уникальный ID этого instance

  db:
    image: postgres:16-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=clearledger
```

```
      - POSTGRES_USER=cl
      - POSTGRES_PASSWORD=${DB_PASS}

volumes:
  storage:
  pgdata:
  backups:
```

---

5. Жизненный цикл документа

```
Клиент загружает файл через UI
        |
        ▼
┌──────────────────────────────────────────────┐
|  API: POST /api/intake (multipart)           |
|                                              |
|  1. Файл → /data/storage/.../uuid_name.pdf   |  Layer 1
|     (immutable, папочная структура)          |
|                                              |
|  2. Парсинг (на контейнере, Python):         |
|     PDF → PyMuPDF → текст + страницы         |
|     Excel → openpyxl → строки + заголовки    |  Layer 1a
|     XML → lxml → структура + поля            |
|     Word → python-docx → текст               |
|     Скан → просто сохранить                  |
|                                              |
|  3. staging.raw_entries ← черновик           |
|     processing_status = 'parsed'             |
|                                              |
|  4. staging.sync_queue ← задача на отправку  |
|     direction = 'to_cloud'                   |
└──────────────────────────────────────────────┘
                        |
                        |
            (когда есть интернет)
                        |
                        ▼
┌──────────────────────────────────────────────┐
|  Наш облачный сервер                         |
|                                              |
|  • AI-классификация (категория, тип документа) |
|  • Извлечение полей (номер, дата, сумма, ИНН) |
|  • Нормализация (контрагенты, справочники)   |
|  • Валидация (суммы, даты, обязательные поля) |
```

```
│  • Дедупликация (по fingerprint + метаданным)   │
│  • Решение: accepted / needs_review / rejected  │
│                                                 │
└─────────────────────────────────────────────────┘
                          │
                          │
                          ▼

┌─────────────────────────────────────────────────┐
│  API: получает результат из облака              │
│                                                 │
│  5. staging.ai_results ← результат обработки    │
│     decision = 'accepted'                       │  Layer 1a
│     normalized_metadata = {...}                 │
│                                                 │
│  6. Если accepted:                              │
│     public.entries ← чистая запись              │  Layer 2
│     public.sources ← метаданные файла           │
│     staging.raw_entries.status = 'promoted'     │
│                                                 │
│  7. Если rejected:                              │
│     staging.raw_entries.status = 'rejected'     │  Layer 1a
│     (файл остаётся в Layer 1, запись —          │
│      только в staging для нашего аудита)        │
│                                                 │
│  8. Клиент видит в UI только public.entries     │  Layer 2
│     Дашборд, таблицы, Excel, 1C — всё отсюда   │
└─────────────────────────────────────────────────┘
```

---

6. Обработка на контейнере клиента (Layer 1a)

Только механический парсинг. Без AI, без нейросетей.

Тип файла	Библиотека	Что извлекает	Ресурсы
PDF	PyMuPDF (fitz)	Текст, кол-во страниц	Минимальные
Excel	openpyxl	Строки, заголовки, листы	Минимальные
XML	lxml	Структура, поля, атрибуты	Минимальные
Word	python-docx	Текст, таблицы	Минимальные
CSV	stdlib csv	Строки, заголовки	Минимальные
Email	stdlib email	Тема, от, тело, вложения	Минимальные
Скан/фото	—	Ничего (только сохранить)	Нет

Требования к машине клиента: Любой VPS/сервер с 1 GB RAM и Docker. Парсинг одного файла — миллисекунды.

---

7. Обработка в облаке (AI)

Тяжёлая, умная работа — только у нас.

Задача	Технология	Что делает
Классификация	Claude API / своя модель	Определяет категорию, тип документа
Извлечение полей	Claude API / regex + ML	Номер, дата, сумма, контрагент, ИНН
Нормализация	Справочники + fuzzy match	"ООО ЛУКОЙЛ" → "ПАО Лукойл" (ИНН 7707083893)
OCR (сканы)	Tesseract / Claude Vision	Текст из изображений
Валидация	Правила + AI	Суммы = НДС + основа? Дата в прошлом?
Дедупликация	fingerprint + metadata	Этот документ уже загружали?
Решение	AI + правила	Принять / на ревью / отвергнуть

---

8. Офлайн-режим

Интернет пропал — система продолжает работать:

Функция	Без интернета	С интернетом
Загрузка файлов	Работает. Файл → Layer 1, парсинг → Layer 1a	То же самое
AI-обработка	Копится в sync_queue	Отправляется пакетом
Просмотр Layer 2	Работает (уже в БД)	Работает
Новые записи в Layer 2	Не появляются (ждут AI)	Появляются после обработки

При восстановлении связи — sync_queue отправляется пакетом, результаты возвращаются, записи промоутятся в Layer 2.

---

9. Безопасность

Контейнер клиента

Угроза	Защита
Доступ к staging	PostgreSQL не торчит наружу. API отдаёт только public schema
Доступ к файлам	Отдаются только через API с JWT
Потеря данных	pg_dump ежедневно + rsync файлов → /data/backups
Наш доступ	SSH с ключом, все действия в audit_log
Атака на контейнер	Минимальный образ, non-root, read-only FS (кроме volumes)

Связь с облаком

Угроза	Защита
Перехват данных	**mTLS** (mutual TLS): клиентский сертификат вшит в Docker-образ + серверный сертификат на облаке
Подмена облака	mTLS certificate pinning — контейнер принимает ТОЛЬКО наш серверный сертификат
Подмена контейнера	Облако принимает ТОЛЬКО запросы с валидным клиентским сертификатом + `INSTANCE_ID`
Утечка данных	В облако уходит ТОЛЬКО `extracted_text` и метаданные, НЕ оригинальные файлы
Наше облако упало	Клиент работает в режиме `grace` (парсинг + sync_queue), AI-обработка возобновляется при восстановлении

Лицензирование и привязка

```
INSTANCE_ID = sha256(ИНН + MAC + hostname + salt)
CLOUD_API_KEY — выдаётся при регистрации, привязан к INSTANCE_ID

При каждом запросе к облаку:
  Header: X-Instance-ID: {INSTANCE_ID}
  Header: Authorization: Bearer {CLOUD_API_KEY}
```

```
  + mTLS client certificate

Облако проверяет:
  1. mTLS сертификат валиден?
  2. CLOUD_API_KEY валиден и привязан к этому INSTANCE_ID?
  3. Подписка активна?
  4. Лимит документов не превышен?
```

Перенос контейнера на другой сервер → `INSTANCE_ID` изменится → требуется перерегистрация.

Heartbeat и деградация

Контейнер отправляет heartbeat каждые **24 часа** на `license.clearledger.ru`:

```
// POST https://license.clearledger.ru/api/heartbeat
{
  "instance_id": "client-npk-001",
  "version": "1.2.3",
  "documents_processed_24h": 47,
  "last_sync": "2026-02-27T10:00:00Z"
}
// Ответ:
{
  "status": "active",
  "next_check": "2026-02-28T10:00:00Z",
  "features": ["case1", "case2", "case3"]
}
```

Режимы деградации (degrade-mode):

Состояние	Условие	Что работает	Что заблокировано
`active`	Подписка оплачена, heartbeat <24ч	Всё	—
`grace`	Нет связи 1–7 дней	Просмотр, экспорт, парсинг в sync_queue	AI-обработка (копится)
`suspended`	Нет связи >7 дней ИЛИ подписка истекла	Только чтение (read-only)	Загрузка, обработка, экспорт
`terminated`	Договор расторгнут	Экспорт данных (30 дней)	Всё остальное

Логика деградации встроена в FastAPI middleware. При `grace` документы парсятся и копятся в `sync_queue` — при восстановлении связи пакетно обрабатываются. При `suspended` данные не теряются, но работа остановлена до оплаты/восстановления связи.

> Полная стратегия защиты IP (7 уровней), монетизация и юридическая защита — MARKETING_AND_IP_STRATEGY.md

Модель ролей

```
admin    → всё (мы, удалённо для поддержки)
owner    → все данные и настройки своего instance
operator → загрузка, просмотр Layer 2, верификация, экспорт
viewer   → только чтение Layer 2
```

Никто из ролей не видит staging. Это внутренний слой системы.

---

10. UI: что видит клиент

Клиент работает **только с Layer 2** (public schema):

Страница	Данные
Дашборд	KPI из public.entries (кол-во, статусы, категории)
Inbox	public.entries WHERE status = 'new'
Таблицы по категориям	public.entries с фильтрами
Детальная карточка	public.entries + файл из Layer 1 через API
Excel/сводные	Экспорт public.entries
1С выгрузка	public.entries WHERE status = 'verified' → XML
Настройки	public.settings, companies, connectors

Индикатор обработки (необязательно): «3 документа в обработке» — count из staging.raw_entries WHERE status = 'parsed' OR 'sent'. Без деталей.

---

11. Что отправляется в облако

Только маскированный текст и метаданные. Файлы и ПДн НЕ уходят.

Перед отправкой контейнер пропускает `extracted_text` через **PII-маскер**: ФИО, ИНН, телефоны, карты, паспорта заменяются токенами `[PERSON_1]`, `[INN_1]` и т.д. Маппинг токен→оригинал (`pii_map`) хранится ТОЛЬКО в контейнере. Три режима privacy: `standard` (критичные ПДн), `strict` (все ПДн + реквизиты), `local-first` (в облако только исключения). Подробнее → MARKETING_AND_IP_STRATEGY.md.

```
// POST https://cloud.clearledger.ru/api/process
{
  "instance_id": "client-npk-001",
  "privacy_mode": "standard",
  "batch": [
    {
      "raw_entry_id": "a1b2c3d4-...",
      "file_name": "TTH_245.pdf",
      "mime_type": "application/pdf",
      "masked_text": "Товарно-транспортная накладная №245 от 25.02.2026,
водитель [PERSON_1], ИНН [INN_1]...",
      "extracted_fields": {
        "page_count": 3
      },
      "company_profile": "fuel"
    }
  ]
}

// Ответ (облако работает с маскированным текстом)
{
  "results": [
    {
      "raw_entry_id": "a1b2c3d4-...",
      "decision": "accepted",
      "classification": {
        "category_id": "primary",
        "subcategory_id": "ttn",
        "doc_type_id": "ttn-gsm",
        "confidence": 0.92
      },
      "normalized_metadata": {
        "docNumber": "245",
        "docDate": "2026-02-25",
        "counterparty": "[INN_1]",
        "amount": "65100.00",
        "currency": "RUB",
        "title": "ТТН №245 от 25.02.2026 — [INN_1]"
      },
```

```
      "metadata_version": "v1",
      "model_version": "cl-classify-v1.2"
    }
  ]
}
// Контейнер подставляет оригиналы из pii_map:
//   [INN_1] → "7707083893" → ищет в НСИ → "ПАО Лукойл"
```

---

12. Технологический стек

Контейнер клиента

Компонент	Технология	Назначение
Web-сервер	nginx	SPA static + reverse proxy
API	Python 3.12 + FastAPI	CRUD, парсинг, sync с облаком
ORM	SQLAlchemy 2.0 + asyncpg	Работа с PostgreSQL
Миграции	Alembic	Автоматические при старте
БД	PostgreSQL 16	staging + public schemas
Парсинг PDF	PyMuPDF (fitz)	Извлечение текста
Парсинг Excel	openpyxl	Строки и заголовки
Парсинг XML	lxml	Структура документа
Парсинг Word	python-docx	Текст из docx
Auth	JWT (PyJWT) + bcrypt	Авторизация
Валидация	Pydantic v2	Типизация API
Бэкапы	pg_dump + rsync + cron	Встроено в контейнер
Контейнер	Docker + docker-compose	Один образ
Фронтенд	React 19 + Vite	SPA, API-клиент

Облачный сервер

Компонент	Технология	Назначение
API	FastAPI	Приём batch-запросов от instance'ов
AI	Claude API	Классификация, извлечение полей
OCR	Tesseract / Claude Vision	Распознавание сканов

Компонент	Технология	Назначение
Нормализация	Python + справочники	Унификация контрагентов, номенклатуры
БД	PostgreSQL	Справочники, модели, статистика
Мониторинг	—	Здоровье instance'ов, очереди, ошибки

---

13. Бэкапы

Автоматические (cron внутри контейнера)

```
# Ежедневно в 03:00
pg_dump clearledger | gzip > /data/backups/db/clearledger_$(date
+%Y%m%d).sql.gz

# Еженедельно — rsync файлов
rsync -a /data/storage/ /data/backups/files/

# Хранить 30 дневных + 12 недельных бэкапов
find /data/backups/db/ -name "*.sql.gz" -mtime +30 -delete
```

Восстановление

```
docker-compose down
gunzip < backup.sql.gz | psql clearledger
rsync -a /data/backups/files/ /data/storage/
docker-compose up -d
```

---

14. Обновления

```
# Одинаково для on-premise и облака
docker-compose pull
docker-compose up -d
# Alembic автоматически применяет миграции при старте API
```

---

15. Порядок реализации

Этап	Что делаем	Результат
1	docker-compose + nginx + PostgreSQL + Alembic + schemas	Контейнер стартует, БД с двумя схемами
2	FastAPI: auth + CRUD public.entries + public.sources	API работает, можно создавать записи
3	Файловое хранилище: upload → папочная структура	Файлы на диске Layer 1
4	Парсинг: PyMuPDF, openpyxl, lxml → staging.raw_entries	Файлы парсятся на контейнере
5	Sync: staging → облако → staging → promote to public	AI-обработка работает
6	Миграция React SPA: API-клиент вместо localStorage	Фронтенд работает с сервером
7	Аудит, роли, rate limiting	Production-ready
8	Бэкапы, мониторинг, health checks	Надёжность
9	Облачный сервер: AI pipeline	Классификация, нормализация
10	Интеграции: 1С, email, коннекторы	Полный функционал